

cs-mail Express Lanes — Implementation Plan

Recipient-Granted Relationship Permission

C-SQD

Draft 0.6 — September 2026

Nonnormative implementation plan. This document records settled design decisions and a build order for *express lanes*, the launch-feature form of the scoped-capability extension that the *cs-mail Protocol Specification* describes. The Rust workspace implements lane grants, admission, lifetime, and domain authentication; the product issuance and operational flows build on those boundaries. Interface sketches below explain the design, while the *Rust Reference Architecture* describes domain ownership. The specification controls admission and request outcomes; the C-SQD financial profile controls the separate member program. The user-facing explanation of the feature is the companion memo, *Express Lanes for Bounded Communication*. The Rust grant API now coordinates lane permission with pending-request settlement; native correspondence enforces lane admission.

Design principles. Before changing the implementation or this design, read the [Rust-domain design principles](#) (`docs/rust-domain-principles.md`). They are binding for cs-mail and identity-model; new work must not regress them.

Current account scope. The current C-SQD product has one identifiable person per account, tied to a verified bank account, with multiple email addresses permitted. Organizational senders in these examples are legacy senders or possible future native protocol participants; they do not imply organizational C-SQD accounts. The [C-SQD domain model](#) controls that product distinction.

1 The Central Observation

A lane grants bounded communication permission without a relationship-request charge. Its admission path authenticates the sender, checks blocking and the signed scope, validates durable content, consumes allowance, and records delivery. It creates no refund or forfeiture. Accepted delivery and permitted follow-ups also move no request value, although a follow-up must join an identified request that is awaiting a decision under current recipient policy. Accepting a request with a lane is a distinct recipient decision: it commits the grant and full request refund obligation together. Later messages use the lane’s ordinary uncharged admission path.

This distinction places lane evaluation at the durable admission boundary. It does not imply that the request kernel runs only when money moves: relationship transitions can change permission without a current charge. Annual billing supplies a separate host-level condition: local service coverage must be funded before new communication. That check does not make a lane a financial object or turn a lane message into a charged request.

2 Settled Design Decisions

1. **Lane admission at the edge.** A lane is evaluated at the admission edge, before the kernel is consulted—mirroring how accepted delivery already avoids the kernel. It uses a bounded, idempotent in-memory model for tests and a durable PostgreSQL coordinator for admitted delivery.
2. **Domain as identity, for legacy senders.** A DMARC-authenticated sending domain resolves to a single synthetic sender protocol identity, so every authenticated address at that domain shares one lane and the per-pair aggregate is preserved.
3. **One lane type, a subject enum.** A lane’s subject is either a native opaque handle or a recipient-granted legacy domain scope. The handle identifies the pair; a separate recipient signature authorizes the complete grant. The same versioned record’s subject can be swapped on adoption.
4. **DMARC verification as evidence-as-data.** Verification happens at the edge and produces a `VerifiedDomain` value. A separately versioned binding joins that proof to the recipient-granted lane domain, so lane evaluation remains deterministic and testable without DNS.
5. **Lifetime on the existing scheduler.** Lanes are time-bound on a sliding window anchored to last activity, bounded by the grant’s hard `not_after`, using the shared `ScheduleTask` infrastructure, `cs_schedules`, and the `cs-mail-worker` lease.
6. **Block and lane are mutually exclusive.** A directed pair is at most one of {blocked, lane-authorized, neither}. Blocking hard-revokes any lane; granting a lane to a blocked pair requires an explicit unblock; unblocking leaves the pair unaccepted without restoring a lane.
7. **Issuance is recipient-authorized.** Lanes may follow address handover, writing first, or acceptance of a relationship request with bounded permission. Selecting a request class does not itself authorize a lane.
8. **Volume and declared scope are enforced.** Each lane carries a maximum message count per interval that admission consumes atomically, a versioned declared-purpose constraint, an optional origin constraint, and the declaration authorities it accepts. Admission compares authenticated declarations exactly. This proves what a sender or gateway declared; providers still cannot inspect encrypted content or certify that the declaration is semantically truthful.
9. **Acceptance and later lane changes are distinct.** Bounded acceptance grants the lane and settles its request atomically. Subsequent renewal, lapse, and revocation do not reopen that settlement. A separate lane-management grant also resolves a pending request.

3 The Admission Decision

Before choosing a permission path, the host checks funded service coverage for the local recipient and any local sender. A remote sender’s provider owns its own coverage check. Expired coverage does not revoke the lane; it prevents new communication until coverage is available again.

Within that service boundary, express lanes add one branch to the question: which authority permits this inbound message? The ordered decision, evaluated for a directed (`sender`, `recipient`) pair at a single canonical boundary, is:

1. if the pair is **Blocked**, refuse (**ContactBlocked**);
2. else if the relationship is **Accepted**, deliver uncharged through the existing accepted-sender path;
3. else if a valid, in-window lane authorizes the pair and the authenticated message declarations match its constraints, deliver uncharged through the lane path;
4. else, if the message identifies a request awaiting a decision, admit a follow-up only if current recipient policy permits it; refusal cannot create another charge;
5. else, if a request is being prepared or awaiting a decision, return its scoped state without creating a new request;
6. otherwise offer an eligible new-request flow, with explicit sender selection of a published recipient request class and authorization before any charge.

The second and third branches share one durable uncharged admission path. A lane admission creates no request charge, moves no ledger value, opens no relationship solicitation episode, and advances no repeated-attempt state—the same properties required of capability-authorized communication. Tests must establish these effects at the storage boundary; avoiding a particular function call is not by itself proof of the invariant.

```
enum AdmissionOutcome {
    Refused,           // blocked pair
    AcceptedPermission, // existing accepted-sender path
    LanePermission,     // new: valid express lane
    RequestFollowup,    // awaiting decision and current recipient allowance
    ExistingRequest,    // resume preparation or report follow-up refusal
    NewRequestOffer,    // eligibility checked; explicit charge authorization needed
}
```

For native mail, the sender signs the complete uncharged admission request, including content reference, declared purpose, origin declaration, optional payload-schema identifier, message validity, capability reference, evidence, idempotency key, protocol version, deployment domain, and intended provider. The same declarations are cryptographically bound to the encrypted content. A legacy SMTP sender cannot supply a native protocol signature: the trusted ingress service constructs and signs the request only after its configured **DmarcVerifier** returns passing evidence, identifies itself as the declaration authority, and uses legacy or unspecified origin unless it has separate authenticated evidence.

A scope mismatch returns **CapabilityScopeMismatch**, consumes no rate allowance, and moves no value. The message may use another path only under its independently valid authority. A pending request cannot be bypassed by buying another charge. Admission after message validity returns **MessageValidityClosed**: it does not settle an submitted request merely because one follow-up message is stale. An initial request still being prepared that cannot be submitted instead follows the full cancellation/refund rule.

4 The Lane Object and Its Store

A lane is a compact recipient-granted record concerning a relationship. It may be scoped by purpose, time, or volume, but is usually not specific to one conversation. It carries no money and references no message content.

```
struct Lane {
    subject: LaneSubject,          // native identity or verified domain
    recipient: ProtocolIdentity,
    purpose: PurposeConstraint,    // matches authenticated declaration
    origin: Option<OriginConstraint>,
    declaration_authority: DeclarationAuthorityConstraint,
    rate_limit: RateLimit,         // enforced atomically at admission
    window: Duration,             // sliding lifetime
    not_after: CanonicalTime,      // signed hard validity bound
    last_activity: CanonicalTime,
    mode: LaneMode,               // Expiring | Persistent
    version: Version,
}

enum LaneSubject {
    Native(ScopedHandle),         // opaque identifier from cs-mail-privacy
    LegacyDomain(DomainIdentity), // recipient-granted domain scope
}
```

The stored lane is created only from a `SignedLaneGrant`. The recipient signature covers the subject, directed protocol pair, purpose and optional origin constraints, accepted declaration authority, rate limit, sliding lifetime, hard validity interval, mode, protocol version, deployment, intended provider, operational key, and grant version. `ScopedHandle` is an opaque identifier, not the signature or the authority by itself.

`DeclaredPurpose`, `OriginDeclaration`, declaration-authority, and schema identifier are constrained, versioned domain values rather than free-form strings. The base purpose vocabulary includes `Unspecified`; future purposes and schemas use namespaced identifiers. Unknown noncritical schemas do not prevent lane delivery, while an unsupported critical schema is refused before allowance is consumed. The selected request class describes the proposed approach; the lane's purpose constraint describes granted permission. Their representations must not silently equate these distinct meanings.

The lane store is a new, compartmentalized access domain: its own database table, credentials, encryption keys, and access role, per the specification's relationship-graph privacy requirements. It is not a column on the relationship aggregate and not part of `protocol_state`. A ledger or telemetry compromise must not yield the list of correspondents a recipient trusts. Ordinary lane transitions remain capability-owned. Acceptance with a lane requires an application coordinator, now implemented, to commit the grant with the protocol-owned request outcome and refund obligation, rather than two independently successful calls.

The storage coordinator enforces one invariant under the relationship aggregate lock: for any directed pair, at most one of {blocked, lane-authorized} holds at a time.

5 Legacy Senders: Domain as Identity

Most transactional senders will not be on *cs-mail* at launch, so the common case is legacy email authenticated by DKIM and DMARC. The inbound edge verifies DMARC under RFC 9989 and produces a `VerifiedDomain`; lane evaluation consumes that value and never performs DNS or signature work itself. This keeps lane logic pure and unit-testable and confines the messy legacy-world cryptography to one edge boundary.

A passing result keeps the RFC 5322 author domain, independent DKIM and SPF pass facts (each with its applicable strict or relaxed alignment mode), and canonical evaluation time as evidence. It does not claim that the verifier exposed a DMARC policy-record or organizational domain when the underlying library did not. Lane identity is instead the canonical IDNA-ASCII domain scope deliberately selected by the recipient’s grant; it is never whichever DKIM or SPF identifier happened to pass. The authenticated author must be that domain or a subdomain of it. That lane domain resolves through a versioned, domain-separated mapping to a *single* synthetic sender protocol identity. Consequently `statements@bank.com` and `alerts@bank.com` are one sender to the system, one lane covers both, and admission still keys to a single `(sender, recipient)` aggregate. A message that fails DMARC alignment to a lane’s domain is simply not on the lane, regardless of its display name or local-part. DMARC resists unauthorized use of the expected domain; the separate expected-domain match rejects look-alike domains.

DMARC says nothing about message purpose, automation mode, or human authorship. The gateway therefore marks the envelope as gateway-declared and uses `LegacyOrUnspecified` origin unless another authenticated mechanism supports a narrower statement. A recipient grant explicitly decides whether a gateway-originated purpose declaration is acceptable. The user interface must not present that choice as content verification; recipient reports, revocation, and blocking remain the response to dishonest use of the declared scope.

The earlier adapter design identifies a dedicated `cs-mail-smtp-gateway` library. It specifies Stalwart Labs’ `mail-auth` 0.12.1 with default features disabled and the minimal Ring/Hickory DNS feature set. The gateway accepts the remote IP, EHLO identity, MAIL FROM (including the null reverse-path case), receiver hostname, raw message, and canonical receipt time. It bounds message size, header size and count, DKIM-signature count, and receipt/system-clock skew before performing DNS-backed verification. Temporary DNS failures remain distinguishable from authentication failure, malformed input, and resource exhaustion for SMTP policy.

6 Native Senders: Pairwise Handles

For a sender already on *cs-mail*, the lane binds to the pairwise-scoped `ScopedHandle` already provided by `cs-mail-privacy`. The recipient signs the full lane grant containing that handle. The provider verifies the grant at admission without learning which organization it authorizes, so the native lane is fully private where the legacy lane must expose a domain. When a legacy sender adopts *cs-mail*, its lane’s subject is swapped from `LegacyDomain` to `Native`, upgrading the lane in place and preserving its history.

Native identity also enables finer control that domain granularity cannot offer. Because a legacy domain is a single identity, block and lane are mutually exclusive at domain granularity—one cannot block a bank’s marketing while keeping a lane for its fraud alerts. A native organization that uses distinct protocol identities for distinct purposes can be blocked on one and lane-granted on another.

7 Lifetime and Renewal

Lane renewal concerns permission, not the annual purchase of service. Billing owns advance collection and the fixed year-long coverage interval; neither changes a lane grant or its hard validity bound. A member allocation is sent as one automatic bank payment with rebate/excess reporting, independently of future renewal. It leaves both purchased coverage and the lane grant intact. See the [C-SQD domain model](#) for financial and account rules.

A lane’s ordinary horizon follows its last authenticated admitted message. Each admitted lane message bumps `last_activity` and reschedules the lane’s horizon; activity can extend that horizon only within the signed hard validity bound, while a dormant lane drifts toward its ordinary expiry.

Two modes cover the range the memo describes:

- **Expiring** (default): at the horizon the recipient receives a gentle re-confirm prompt rather than a silent deletion. The window is set per sender.
- **Persistent**: the same renew-on-activity behavior, but the horizon keeps the lane active and emits a review reminder instead of lapsing it. The signed `not_after` remains a hard stop. This is the mode for a bank’s fraud alerts or a hospital’s results, where a silently lapsed lane would be costly.

Both modes reuse the leased `cs_schedules` plus `cs-mail-worker` machinery: a scheduled task fires at `last_activity + window` and either lapses the lane or emits a review effect through the outbox. The generic `ScheduleTask` and `ScheduleChange` types live in `cs-mail-primitives`; no second lease substrate is required.

8 Block and Lane Interaction

Block and lane are opposite ends of one recipient control, so the model treats them as mutually exclusive rather than as independent flags. The two directions are deliberately asymmetric:

- **Blocking hard-revokes the lane, unconditionally.** Block is the most restrictive state and a deliberate protective action, so it tears down any lane in the same commit that records the block. After a later unblock the organization does *not* regain its lane; it returns to an unaccepted sender, subject to preserved request history and any other independently valid authority.
- **Granting a lane never silently unblocks.** Because issuance is one-tap or automatic, an issuance flow must not silently defeat a block the recipient deliberately set. A grant targeting a blocked pair surfaces the block and requires an explicit unblock-and-allow confirmation.

This mutual-exclusion invariant also resolves the coordination question raised by placing lanes outside the kernel. Block state lives in the relationship aggregate; the lane lives in the edge store. Rather than reconciling two independent truths, blocking is one state transition that revokes the lane in the same commit, and lane admission reads the pair’s block state transactionally—by taking the relationship aggregate’s row lock, or by consulting a minimal transactional block flag—so a block that commits first is always observed. Domain-as-identity is what makes “the pair” well defined: blocking `bank.com` and holding a lane for `bank.com` concern the same directed identity pair.

The reference implementation chooses the aggregate row lock. A future split-store deployment must preserve the same linearization contract through a narrowly authorized transactional coordinator.

9 Issuance

A lane is granted only by a deliberate recipient act:

- **At address handover** (checkout, signup, booking). The client records the web origin it is on as the expected sending domain, then completes the grant on the first DMARC-authenticated message from that domain, showing a one-tap confirmation that names the *authenticated* domain rather than the display name. Seeding the expectation from the origin turns “which sender?” into a match and lets the recipient catch a look-alike domain at grant time.
- **On replying first.** Writing to an organization auto-opens a return lane, since the recipient has plainly invited a reply.
- **On accepting a request with a lane.** The recipient grants bounded relationship permission and the full $C + S$ refund obligation in one decision. The request’s class supplies context, not standing permission.

A cold sender selects from the recipient’s published request classes. Acceptance may grant standing permission or a lane; either returns the full charge. Rejection and expiry retain their existing allocations. A lane grant during preparation cancels submission and voids or refunds the full charge. A submitted request still pending receives the full refund even after its deadline. Terminal settlements remain fixed; future access follows the lane without a new bond.

10 Changes by Crate

- **cs-mail-capabilities** holds the lane “book”: the lane record, subject enum, purpose and origin constraints, declaration- authority rules, and idempotent grant/renew/revoke transitions, with no ledger coupling in ordinary lane operations and the block/lane mutual-exclusion invariant. Its admission check performs exact versioned declaration matching before consuming allowance.
- **cs-mail-adapters** defines the async, library-neutral SMTP authentication request, typed failures, and versioned evidence model, alongside the existing outbound downgrade route.
- **cs-mail-smtp-gateway** implements that boundary with pinned Stalwart **mail-auth**, minimal crypto/DNS features, resource limits, and receipt-time skew checks.
- **cs-mail-storage-postgres** gains a migration for the compartmentalized lane table under its own access role and keys, plus the transactional block/lane coordination. The bounded-acceptance extension must also commit the lane, request outcome, refund work, and receipt together.
- **cs-mail-primitives**, **cs-mail-worker**, and **cs_schedules** supply shared task types and the existing lease for lane horizon tasks (lapse or review); no new scheduling substrate.
- **cs-mail-privacy** supplies **ScopedHandle** as the native pair identifier; **cs-mail-capabilities** supplies the separately signed authority.
- **cs-mail-primitives** defines constrained purpose, origin, declaration-authority, message-validity, schema, and extension- criticality values without acquiring workflow or policy behavior.

- **cs-mail-content** and **cs-mail-wire** bind the declarations, capability reference, and validity time to encrypted content and provide canonical, versioned representations. Unknown noncritical schemas remain opaque; unsupported critical schemas fail before admission.
- **Service and capability layers** add authenticated ingress and domain-separated, versioned signing representations for grants and uncharged admission. Provider admission policy consumes envelope-visible facts only; recipient presentation policy remains in the client and has no settlement authority.
- **cs-mail-billing** owns funded service coverage; **cs-mail-storage-postgres** checks it at the host admission boundary. Lane grants and renewal remain in capabilities, and annual eligibility and allocations remain in finance. None is inferred from another’s activity.
- **cs-mail-protocol::transition** (the settlement kernel) owns request outcomes. The application coordinates its bounded-acceptance effects with the capability grant; the capability domain does not choose refund policy. Checked publications allow at most eight user-defined classes; signed terms retain the selected class and its fixed financial terms.

11 Build Order

Each lane stage is independently testable after the revised request baseline is established. References to earlier implementation status are not conformance evidence for this revision.

1. **Native lanes, end to end.** The lane book, the admission branch, grant and revoke, and the block/lane mutual-exclusion invariant with its transactional coordination established the permission layer and its hardest correctness property first.
2. **Lifetime and renewal.** Sliding window, the two modes, and the scheduled lapse/re-confirm tasks on the existing worker lease.
3. **Legacy senders.** The inbound DMARC verifier, domain-to-identity resolution, and the legacy subject, with independent SPF and DKIM evidence and expected-domain mismatch enforcement.
4. **Issuance flows.** Record the expected domain at address handover, confirm the first authenticated message, authorize a return lane when the recipient writes first, and implement atomic acceptance of a request with a lane.
5. **Authenticated declaration enforcement.** Bind constrained purpose and origin declarations to native content and uncharged admission signing bytes, add explicit gateway authority, and reject mismatches without consuming lane allowance.

12 Open Questions and Risks

- **Split-store linearization.** The reference implementation uses the relationship aggregate row lock. Any future split-store deployment must preserve that ordering contract with a narrowly authorized coordinator.
- **Renewal signal.** An authenticated, admitted lane message renews the sliding horizon using monotonic canonical time. Decide separately whether recipient reads or replies should count.

- **DMARC operations.** The reference selects pinned Stalwart `mail-auth` and keeps DNS out of lane logic. A deployment must still operate resolver policy, map typed failures to SMTP replies, monitor latency, and keep its system clock within the configured receipt-time skew. Senders without a DMARC pass receive no legacy lane; a weaker grant would be a separately named and reviewed feature.
- **Per-subject retention.** Native lane subjects are opaque; legacy subjects expose a domain. Retention and audit rules differ per subject and must be encoded in the store’s retention classes.
- **Issuance as a surface.** Rate-limit issuance confirmations and prevent the lane store or its prompts from becoming an enumeration oracle.
- **Declaration honesty and metadata.** A matched declaration is not proof that ciphertext has the claimed meaning. Keep the vocabulary coarse, retain it only as long as required for admission and audit, expose no recipient policy beyond actionable constraints, and make report, revoke, and block easy.

13 Request and Member-Program Integration

Add conformance stories for a lane message while a relationship request is open, lane revocation followed by an allowed or refused request follow-up, and a block racing both paths. Add extension scenarios for class-bound quotes and bounded acceptance: one full refund obligation and one lane commit, no standing acceptance, no partial outcome on conflict or failure, and no duplicate effects on retry. Later lane expiry or revocation must not reopen that settlement. Lane expiry cannot extend a request deadline, and an unsupported declaration cannot silently start a charged request.

An admitted lane message may renew the lane’s activity horizon. It must not count automatically as intentional member activity, contribute forfeitures, or affect an annual allocation. These programs may reuse scheduling and scoped identifiers, but their authorities and financial effects remain separate. Add coverage tests for an otherwise valid lane after service expires, and for restored delivery within the grant’s remaining validity after funding finalizes. A member’s earned allocation remains payable regardless of service renewal or lane renewal.